A Practical Intro to HPC

Skillshare session, 10 March

Mark Krumholz, 6 March 2026

Outline

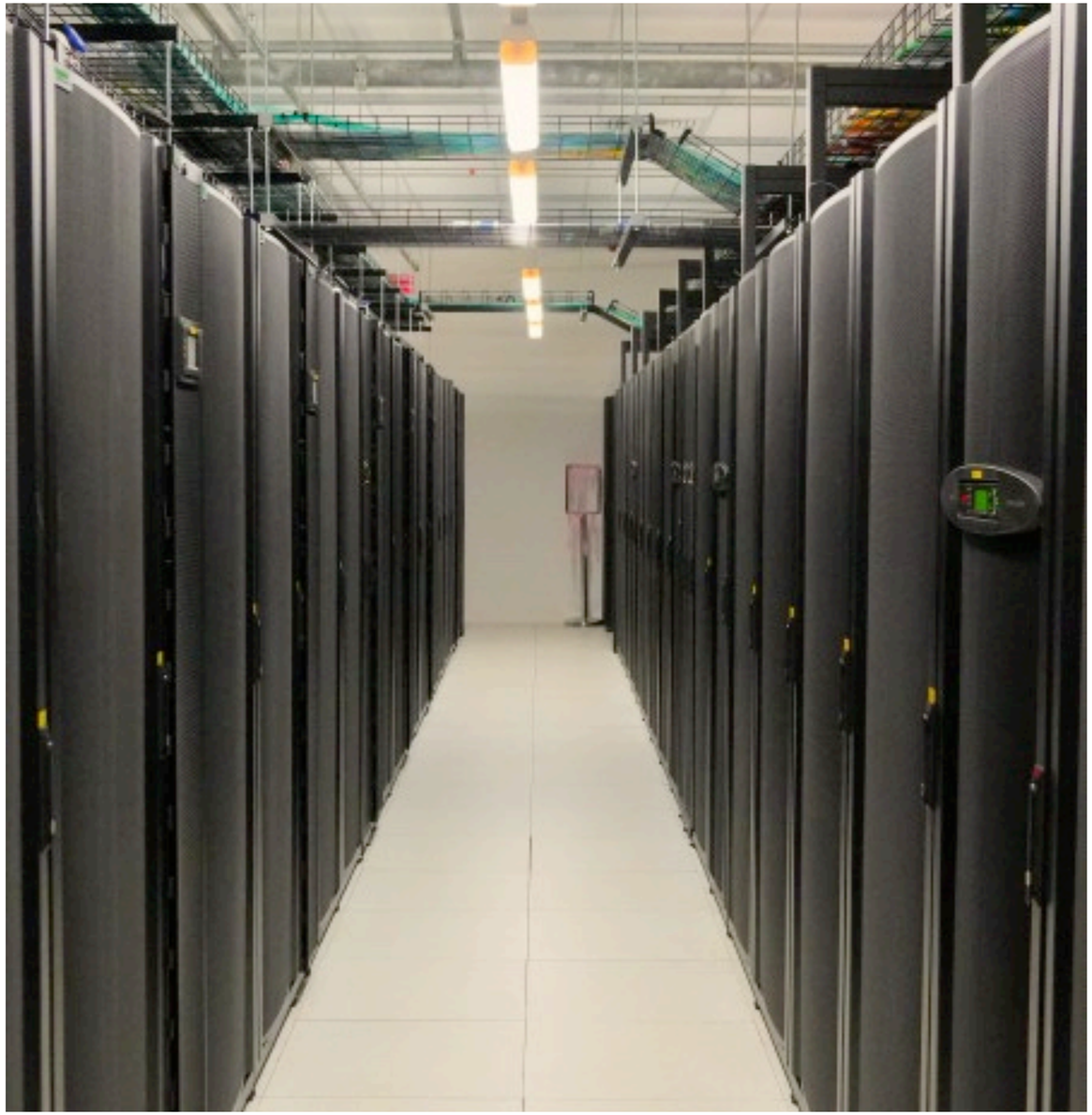
Skillshare 10 March

- A brief overview of HPC systems: nodes, queues, filesystems
- Foundations of parallel computing: memory models, parallel efficiency
- How to get access to HPC
- How to find out more
- A practical tutorial

HPC systems

A brief introduction

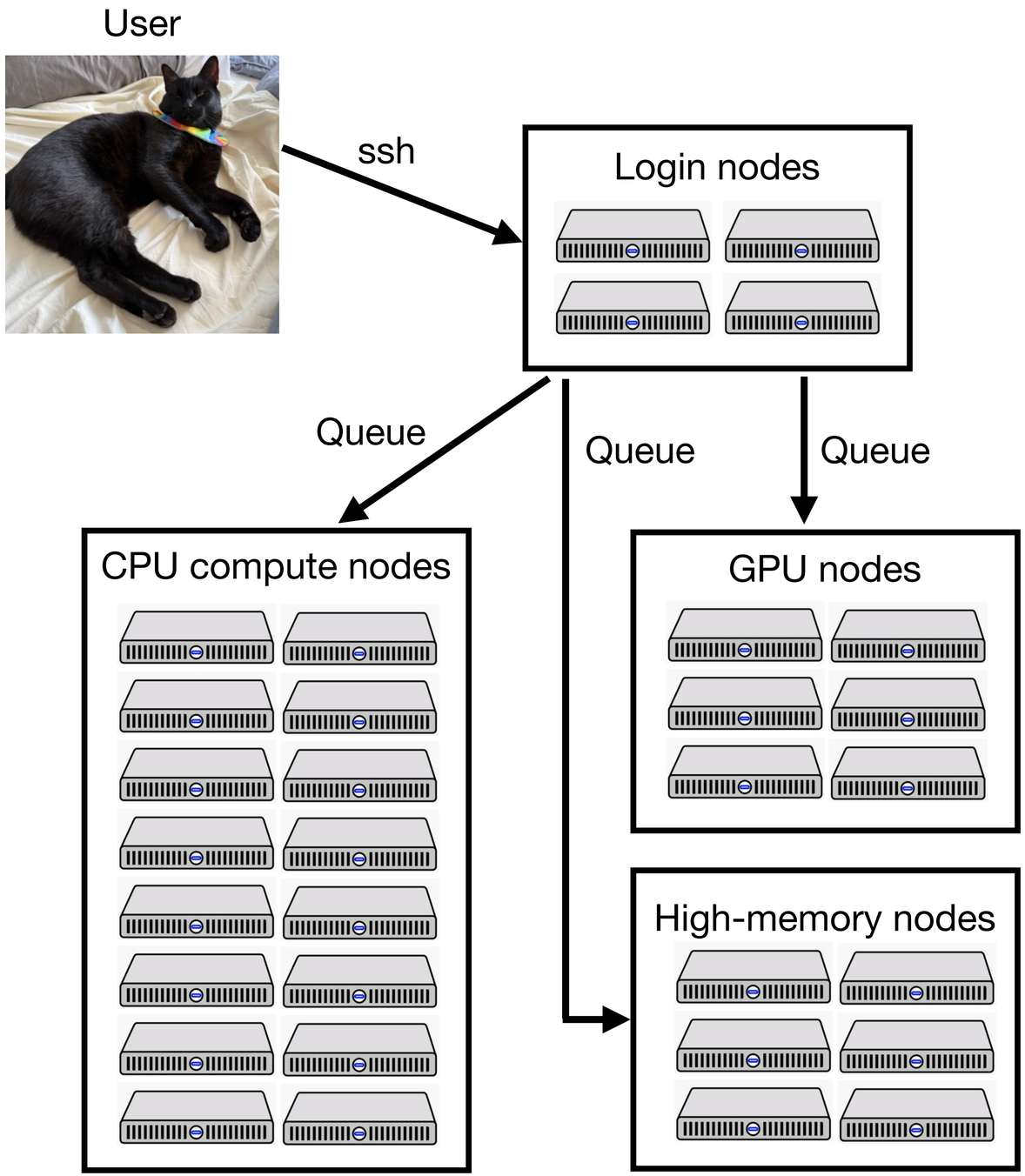
- An HPC system consists of a collection of *nodes*
- Each node is an independent computer with its own processors, RAM, disk, etc. installed as a blade in a rack
- Nodes are connected via network — almost always something faster than standard ethernet
- System sizes range from ~10 to ~10,000 nodes



Types of nodes

Roles of nodes in the system

- Login (or head) nodes: user interface, interactive tasks like file editing, compiling, submitting jobs
- Work (or compute) nodes: do calculations
- There may be multiple types of work nodes with different hardware for different tasks (e.g., CPU vs. GPU computing, memory-intensive, I/O-intensive)
- Usually it is only possible to connect to the login nodes directly — all other nodes are accessed via a queue



Example node configurations

From Gadi at NCI

- normalsr (CPU compute nodes):
  - 2 x 52-core Intel Xeon Platinum 8470Q (Sapphire Rapids) 2.1 GHz CPUs
  - 512GiB RAM
  - 400 GiB local SSD disk
- hugemem (high-memory nodes):
  - 2 x 24-core Intel Xeon Platinum 8268 (Cascade Lake) 2.9 GHz CPUs
  - 1.5 TiB of Intel Optane DC Persistent Memory with 384 GiB DRAM
  - 1.5 TiB local SSD disk per node
- gpuhopper (GPU compute nodes):
  - 2x 24-core Intel Xeon Gold 6542Y 2.9GHz CPUs per node
  - 1 TiB RAM per node
  - 4x NVIDIA H200-SXM5-141GB per node
  - 1.8 TiB local SSD disk

How to do work on an HPC system

Queue-mode computing

- Only use the login nodes for light, interactive tasks (editing files, compiling code, transferring small amounts of data, very quick and simple data visualisation, etc.)
  - Running big jobs on the login node is strongly discouraged, and may have consequences like your job being summarily killed and angry emails from sysadmins
- For all other tasks use compute nodes accessed through a *queue scheduler*
- Basic steps (all carried out from the login nodes):
  1. Write a script describing both the resources needed (how many nodes, what type of node, for how long, etc.) and the tasks to run
  2. Submit your script (either “qsub myjob.sh” or “sbatch myjob.sh” depending on whether the system is using the pbs or slurm scheduler)
  3. Wait for job to run, look at results
- It is also possible to request nodes for interactive use, for example to run a Jupyter notebook; we will discuss how to do that in a moment

Structure of a job submission script

This example uses the pbs scheduler, but slurm is similar

Lines that start with # are scheduler instructions

First line specifies with shell to use

#PBS lines describe the job:

- -N: name of job
- -P: which project to charge
- -q: which queue (nodes) should run the job?
- -l walltime: for how long is the job reserving resources?
- -l mem: how much memory (across all nodes) is the job allowed to use?
- -l ncpus: how many CPUs are requested? (must be divisible by of # of CPUs/node)

Remaining lines are the commands to be carried out, treated exactly as if you had typed them in the terminal

```
#!/bin/bash
#PBS -N Terzan5_criptic
#PBS -P jh2
#PBS -q normalsr
#PBS -l walltime=24:00:00
#PBS -l mem=2000GB
#PBS -l ncpus=416
#PBS -l wd

export OMP_NUM_THREADS=13
export GOMP_CPU_AFFINITY=0-103

mpirun -np $(( $PBS_NCPUS / $OMP_NUM_THREADS )) --map-by node:
PE=$OMP_NUM_THREADS --rank-by core ./criptic criptic.in >& cri
ptic.log
```

Job accounting

Sharing the machine

- Job wall time is generally limited to some maximum that depends on the queue; if you need to run for longer than this, use checkpoints
- Most systems jobs require a project code to charge for the job
- Jobs are billed in “service units”, calculated as (# CPUs / node) x (# nodes used) x (# hours) x (cost factor for queue); for cost factor = 1, SU = number of CPU-hours used
- Billing is by whole node, even if you don’t use all the processors on the node
- Some smaller systems do not use accounting (e.g., RSAA nodes on Gadi, mozzie, OzStar/NT); on these, be courteous and share

Interactive jobs

When you want to compute in real time

- It is also possible to request nodes for interactive use; on PBS systems, the command is “qsub -I”, but you can google the full syntax
  - When you issue this command, you are dumped into a shell on one of the compute nodes you requested, as if you had ssh’ed into it
  - You’re still requesting through the queue, so you might have to wait for a node to become available
  - On many systems there is an “express” queue you can use to minimise wait time, which allows faster access but only short sessions (1-2 hours)
- Use interactive jobs to debug, do visualisation in a Jupyter notebook, etc.
- At the end we’ll do a practical exercise on running in a interactive job in a Jupyter notebook

Filesystems on HPC

Multiple file systems for different uses

- “Home” filesystem (/home on Gadi):
  - Intended for source code, scripts, small files that will be kept for a long time
  - Small quota, backed up
- “Scratch” or “work” (/scratch on Gadi):
  - Intended to hold large data sets or the outputs of large computations
  - Fast connections to nodes for quick I/O
  - Large quota, but may not backed up, may be purged periodically
- “Data” or “storage” (/g/data and mdss on Gadi):
  - Long term storage, lots of space, but access is by proposal or cash per GB
  - Can be disk, tape, or both — tape is very cheap, but access is not instantaneous

Foundations of parallel computing

A *very* short introduction

Memory models in parallel computing

Three types of parallelism

- *Embarrassingly* parallel: large number of independent calculations that can each be run on a single processor with no communication between them
  - Examples: Monte Carlo realisations, radio data processing, but not things like hydro, large N-body simulations
- *Shared memory*: different processes can all access the same memory, so can communicate and coordinate by updating memory
  - Simpler to program, but limited to running on a single node
  - Many software implementations (e.g., threads, OpenMP)
- *Distributed memory*: different processes do not share memory, information must be exchanged by explicit sending of messages
  - Can run across multiple nodes, so the only way to do truly large-scale computing
  - Most common software implementation is message passing interface (MPI), but others exist

Parallel efficiency / parallel scaling

The metric by which we judge parallel applications

- Measures the computer resources required to solve a problem in parallel versus those required to solve it with a single processor
- To understand how efficiency is defined, it is important to distinguish different measures of time:
  - Wall time: how long did the calculation take as measured on your watch (or phone, or wall clock)?
  - Compute time: equal to wall time multiplied by the number of processors (or some other resource) used, has units like CPU-hours or SU
- Ideally using more processes reduces the wall time while leaving compute time unchanged; parallel efficiency is defined relative to this ideal

Weak and strong scaling efficiency

Definitions and motivations

Weak scaling

- Increase CPU count and problem size, keeping work per CPU fixed
- Ideal outcome: wall time fixed, compute time ~#CPUs

Strong scaling

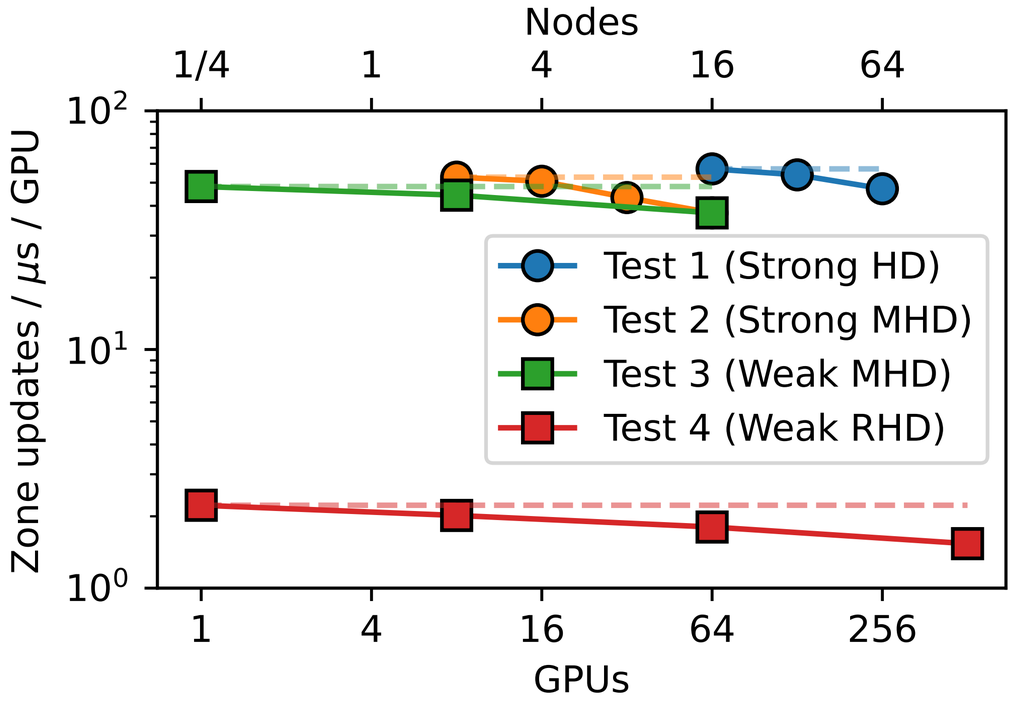
- Keep problem size fixed while increasing CPU count
- Ideal outcome: compute time fixed, wall time ~1 / #CPUs

- In either case, efficiency defined as ideal time / measured time
- Strong scaling harder to achieve than weak scaling, and strong scaling will always become poor for sufficiently small problems

Testing efficiency

A practical guide

- Testing done by running fixed problem at range of CPU counts, often excluding the cost of I/O and startup
- For weak scaling, problem size is increased with CPU count, for strong it is kept fixed
- “Good” efficiency depends on application, but rule of thumb:
  - Embarrassingly parallel codes should achieve near 100%
  - Other memory models should aim for >70%; if you are getting <50%, you’re using too many CPUs



Writing your own parallel code

A quick guide

- First step is to determine which type of parallelism you want — what does your problem demand?
- For embarrassingly parallel, can just write job script that launches a bunch of independent jobs, or do something similar with, e.g., a Python script
- For shared memory, first try tools like Python multiprocessing or C/C++ OpenMP loop decorators, but may require tuning to get good performance
- For distributed memory, need careful design; generally experts only
- *Always* test your parallel efficiency before deploying code at scale!

Why doesn’t my code scale well?

Sources of inefficiency

- Resource contention: multiple cores trying to access the same memory, getting in each other’s way
- Load balancing: work not evenly distributed, so some processors are sitting idle waiting for others to finish
- NUMA issues: turns out that “memory” is a real physical object, and not all parts of it can be accessed equally fast
- Communication costs in distributed memory: transmitting data between nodes takes time, and also incurs latency (time to initiate communication — typically ~1 μs for HPC interconnects, ~50 μs for ethernet, which is why HPC systems don’t use standard ethernet)

HPC access

Starting with: do I need HPC?

- Always start from the problem: what do you need to compute, and why?
- Questions to consider next:
  - Will this take too long on a desktop / workstation?
  - If yes, is this because my code is inefficient? It’s usually easier to fix code than to set up on an HPC system
  - If the code is efficient and it still can’t be done on a workstation, then consider going to HPC

Planning to use HPC

A step-by-step guide

- First step: estimate how much time you need
  - Test on small problems / cases to get a sense of cost per unit or work
  - Extrapolate to your intended calculation, in both CPU time and wall time
  - If you are considering a big multi-node calculation, be sure to test parallel efficiency and factor inefficiency losses into your estimate

- Second step: choose your target system based on your needs
  - Small: ~days - weeks wall time, ~10k-100k CPU-hours: mozzie, RSAA Gadi nodes
  - Medium: weeks - ~month wall time, ~few hundred k CPU-hours: OzStar/NT at Swinburne
  - Large: >1 month wall time, 1M+ CPU-hours: ASTAC, NCMAS, ANUMAS
  - For reference: 24 hours of running on one of the rsaa nodes on Gadi is 2.5k CPU-hours, 1 solid week on 8 nodes is 140k CPU-hours

Available resources

Small-scale projects

- RSAA has two small (~10 node) HPC clusters: mozzie, attached to the RSAA network, and the rsaa and rsaagpu queues on Gadi at NCI on campus
- Anyone with an account on the Stromlo machines automatically has access to mozzie — just log into the head node at mozzie.anu.edu.au
- The Gadi queues are also open to anyone working at Stromlo, but you need to request access — email me (mark.krumholz@anu.edu.au); the rsaa queue goes to CPUs, the rsaagpu queue to nodes with 4 Nvidia A100 GPUs
- Both systems use pbs scheduling for job submission, and do not use accounting, so be courteous and share — be sure you are on the rsaa slack so people can message you about usage

Available resources

Medium-scale projects

- All astronomers in Australia can automatically access the OzStar machine and its successor Ngarru Tindebeek (NT) at Swinburne
- No / minimal application required for moderate usage
- Machine has 150 CPU nodes and 22 GPU nodes; scheduling uses slurm
- No accounting, but queue wait times mean that in practice one cannot push through large projects
- Can also apply for priority time through ASTAC (next slide)

Available resources

Large-scale projects

- Larger projects must apply via competitive process to get time on one of the two national supercomputers: Gadi at ANU, and Setonix in WA
- Access programs:
  - ASTAC: AAL Supercomputer Time Allocation Committee; runs twice per year, can ask for up to 4 MSU on Gadi, as well as time on OzStar
  - NCMAS: National Computational Merit Allocation Scheme; runs once / year, up to ~20 MSU, minimum 1 MSU request; only faculty and independent postdocs can be lead CIs
  - ANUMAS: ANU Merit Allocation Scheme; extra access to Gadi for ANU staff only, similar size and eligibility rules to NCMAS
- Note that these programs are still small by the standards of what is available internationally — the largest NCMAS awards are smaller than the minimum request on the larger programs in the US, EU, and Japan!

How to write a good proposal

Tips for success

- First and most important, talk to someone who has gotten time before and get a copy of their proposal to use as a guide
- Second, proposals are judged on both scientific merit and technical aspects — often with more weight on the later due to the review process
- Technical facets include:
  - How well the time requirement is justified
  - Demonstration of performance (particularly parallel scaling)
  - Evaluation of capabilities of the code

Where to get more information

Follow-up links

- Usage guides for big machines:
  - Gadi: https://opus.nci.org.au/spaces/Help/pages/236880325/Gadi+User+Guide
  - Setonix: https://pawsey.atlassian.net/wiki/spaces/US/pages/51925434/Setonix+User+Guide
- Access program links:
  - OzStar/NT: https://supercomputing.swin.edu.au/
  - ASTAC: https://astronomyaustralia.org.au/committees-astronomy-australia-ltd/astac/
  - NCMAS / ANUMAS: https://ncmas.nci.org.au / https://anumas.nci.org.au/